

Mecanismo de Atualização Dinâmica do Parser no GenESyS

Prévia para apresentação em simpósio

Vicente Cardoso dos Santos Ricardo Henrique Medeiros Pereira Eduardo Bischoff Grasel

Universidade Federal de Santa Catarina
INE5425 – Modelagem e Simulação

1. Apresentando o projeto

Problema, objetivo e visão geral.

2. Desenvolvimento

Contratos, lazy reload, geração, compilação e runtime.

3. Testes

Validação do fluxo completo e das falhas.

4. Resultados

Demonstração, evidências e impacto.

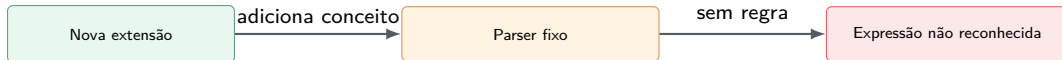
Problema: Modelo Extensível, Parser Fixo

Modelo

- Recebe plugins.
- Ganha novas definições de dados.
- Pode incorporar novas funções conceituais.

Parser original

- Depende de Bison/Flex.
- Reconhece tokens e regras já compilados.
- Não acompanha automaticamente cada plugin.



Objetivo

Objetivo central

Permitir que extensões declarem mudanças de linguagem e que o GenESyS atualize o parser automaticamente.

Declarar

Plugin informa tokens, regras e produções.

Agregar

Kernel coleta contribuições do modelo.

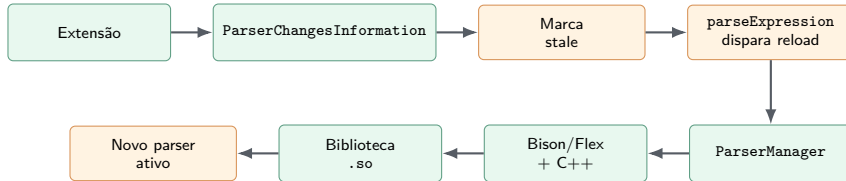
Gerar

Bison/Flex e C++ produzem nova biblioteca.

Conectar

Runtime troca o parser ativo com segurança.

Visão Geral da Solução



Ideia-chave

A extensão declara; o kernel gera e conecta; o modelo usa o parser atualizado.

Contratos Aproveitados e Completados

Já previsto

- `ParserChangesInformation`
- `_getParserChangesInformation()`
- `ParserManager`
- Marcadores em Bison/Flex

Implementado

- Campos léxicos e `hasChanges()`
- Agregação de mudanças
- Geração da `.so`
- Conexão via `dlopen/dlsym`

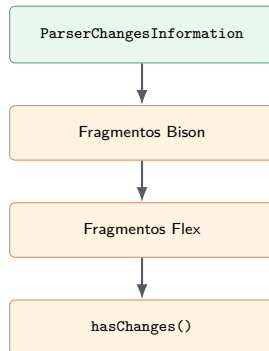
Decisão

Completar a arquitetura existente, preservando os arquivos-base como fonte de verdade.

ParserChangesInformation

Pacote de fragmentos que a extensão entrega ao kernel.

- Includes e tipos semânticos.
- Tokens e produções Bison.
- Regras e literais Flex.
- `hasChanges()` evita recompilação vazia.



Lazy Reload no Ciclo do Modelo

Na inserção

`ModelDataManager::insert()` detecta mudanças e marca o parser como stale.

No primeiro uso

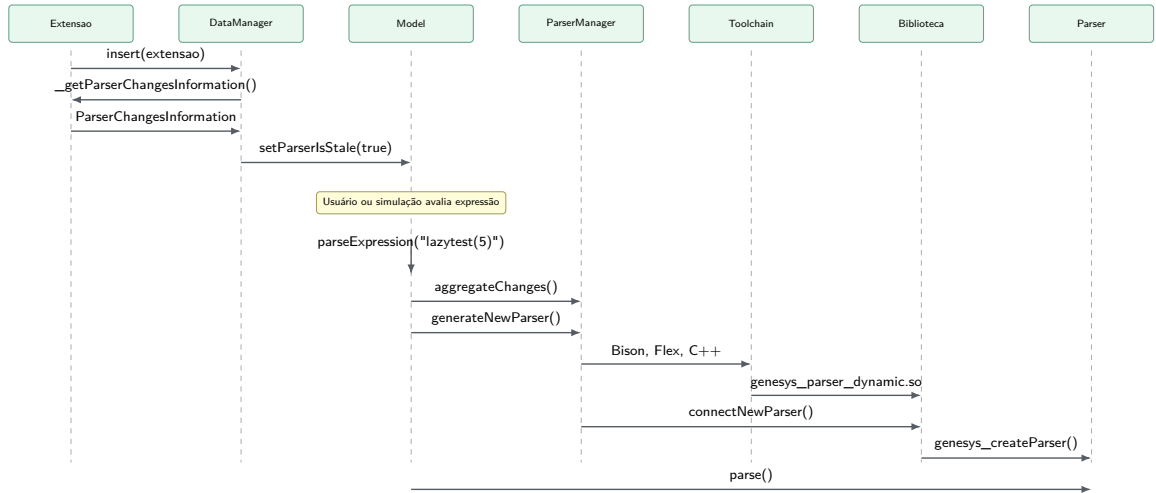
`Model::parseExpression()` sincroniza a linguagem antes de avaliar.



Por que lazy?

Evita recompilações repetidas quando várias extensões são inseridas antes de qualquer avaliação.

Diagrama de Sequência: Lazy Reload



Geração do Parser

- Arquivos-base continuam sendo a fonte de verdade.
- Fragmentos são injetados em marcadores planejados.
- Arquivos modificados são gerados em diretório de trabalho.

Mapeamento

Tokens e produções entram no Bison; regras léxicas entram no Flex.

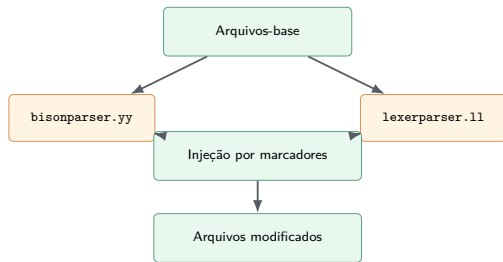
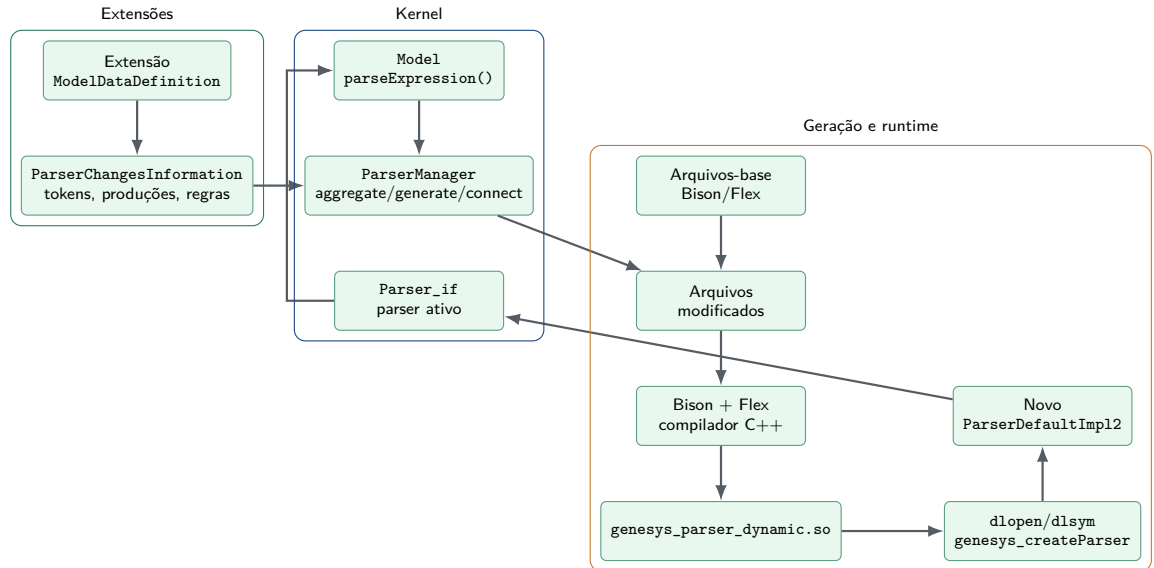


Diagrama de Componentes



Compilação e Carregamento em Runtime

Toolchain

- 1 Bison gera parser C++.
- 2 Flex gera scanner C++.
- 3 Compilador cria .so.

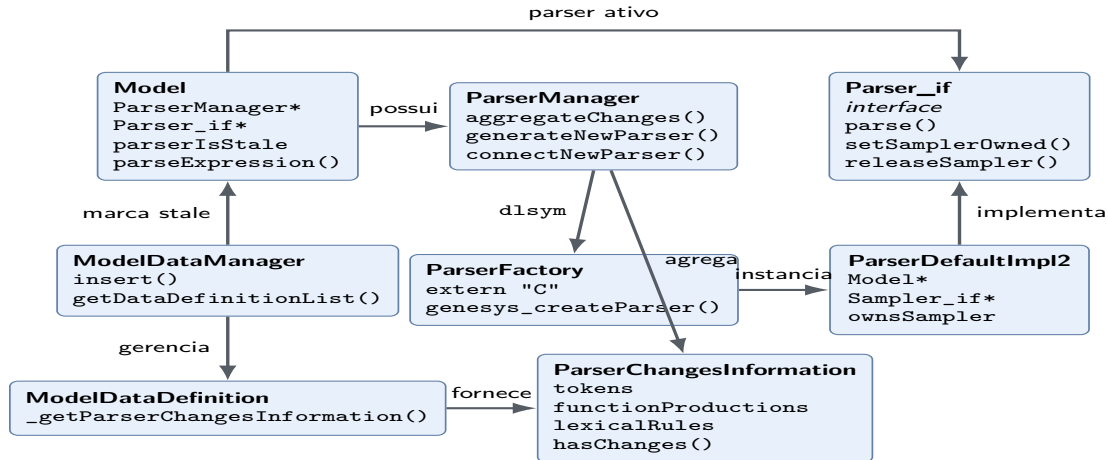
Runtime

- 1 `dlopen()` abre a biblioteca.
- 2 `dlsym()` encontra a factory.
- 3 `genesys_createParser()` instancia o parser.

Fronteira C

`extern "C"` evita problemas de *name mangling* e fornece um ponto estável de criação.

Diagrama de Classes



Substituição Segura do Parser

- Parser antigo deve continuar válido se o reload falhar.
- `Sampler_if` precisa sobreviver à troca.
- Ownership evita vazamento e dupla liberação.

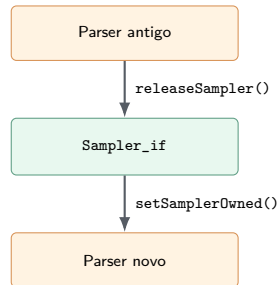
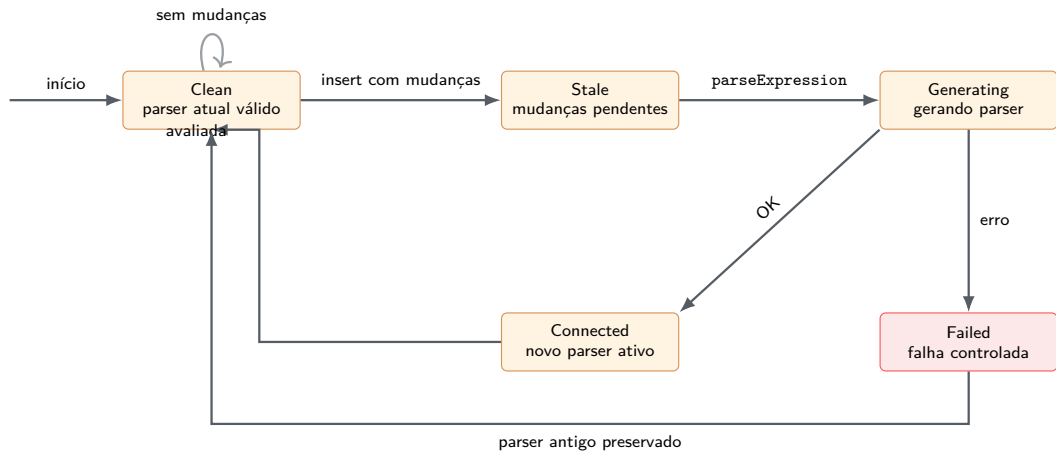


Diagrama de Estados



Contrato

- Campos vazios.
- Armazenamento.
- `hasChanges()`.

Integração

- Extensão nova.
- Lazy reload.
- Múltiplas extensões.

Falhas

- Biblioteca ausente.
- Mudança inválida.
- Parser antigo preservado.

Evidências de Teste

Fluxo principal

Extensão adiciona função

Pipeline completo: declara, gera, compila, conecta e avalia.

Atualização sob demanda

Reload na próxima expressão

Reload automático na primeira chamada a `parseExpression`.

Múltiplas extensões

Agregação de contribuições

Combina contribuições de mais de uma extensão.

Falha controlada

Parser antigo preservado

Falha sem inutilizar o parser anterior.

Biblioteca ausente

Diagnóstico de conexão

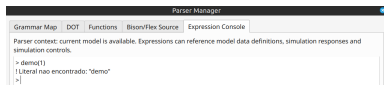
Diagnóstico quando a biblioteca dinâmica não existe.

Ownership

Transferência do sampler

Transferência segura do `Sampler_if`.

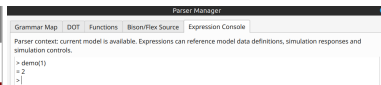
Resultado Observado: Demonstração Demo



Antes: `demo()` não reconhecida



Extensão declarada no modelo



Depois: parser atualizado

Leitura do resultado

A função não existia na linguagem base; a extensão declarou as mudanças; o parser passou a reconhecer a expressão.

Resultados do Projeto

- Contratos previstos passaram a formar um fluxo funcional.
- Plugins podem declarar alterações léxicas e sintáticas.
- O parser é regenerado sob demanda com Bison, Flex e C++.
- A biblioteca gerada é carregada em runtime por uma factory C.
- Falhas preservam o parser anterior e retornam diagnósticos.

Mensagem final

A linguagem de expressões do GenESyS passa a acompanhar dinamicamente as extensões carregadas no modelo.

Obrigado!

Tema

Mecanismo de atualização dinâmica do parser no GenESyS.

Perguntas?